

AI Usage Note: Schema-Aware Test Data Generator

Prompt and Tool Mapping

AI Tool	Prompt (Input)
Claude	1. Claude for planning Project Title: Schema-Aware Test Data Generator You are building a tool that automatically creates mock data for a database. It isn't just generating random text; it understands how database tables are structured and connected. 2. The Problem: "Test environments need realistic data tied to FK relationships. "When developers test an application, they need mock data that looks real (like actual names, emails, and dates instead of "test1", "test2"). Crucially, this data must respect Foreign Keys (FK). For example, if you generate a row in an orders table, the user_id must actually exist in the users table, or the database will throw an error. 3. The Implementation: "Reads schema DDL → LLM + Faker generates referentially-consistent rows across all tables. Outputs SQL inserts + sample CSVs." This is your actual development checklist: Input: Your tool needs to parse a DDL file (the SQL file containing CREATE TABLE statements) to map out the tables, columns, data types, and primary/foreign key constraints. Core Logic: * An LLM (Large Language Model) reads the context of the column names to understand what kind of data belongs there (e.g., recognizing that a column named u_nm means "username"). Faker (a library used to generate realistic fake data like names, addresses, phone numbers) is triggered by that context to generate the actual values. The engine must track dependencies so it populates parent tables before child tables, ensuring referential consistency. Output: The tool should export the generated data as ready-to-run.sql insert scripts and as.csv files. 4. Key Feature: "Constraint-aware generation respecting FKs." Your main technical challenge here is handling the relational graph. The application must process the database schema in a specific topological order (e.g., generating users first, then products, and finally orders which links both) so that no foreign key constraints are violated during insertion. Suggested Next Steps to Start Building: 5. Pick your tech stack: Python is highly recommended for this since it has excellent SQL parsing libraries (sqlglot or sqlparse), great LLM SDKs, and a robust Faker library. 6. Draft a simple schema: Start with just two linked tables (e.g., users and posts) to test your DDL parsing logic. 7. Design the prompt: Create a

AI Tool	Prompt (Input)
	prompt template that passes a table definition to the LLM and asks it to map each column to a specific Faker provider. i need this plan file to feed to my local coding agent so i complete the project :)
Antigravity	Implementation & Refinement (antigravity) Set up the core application structure by creating the FastAPI backend, building the frontend with vanilla JavaScript, and adding API endpoints to upload and process database schemas. Add a token usage tracker below the test data generator that shows the exact number of input and output tokens used for each LLM request. Improve the table navigation experience by enabling horizontal scrolling for table tabs, making it easier to work with large schemas containing many tables. Adjust the Flexbox layout so that the table tabs remain perfectly aligned with the "Generated Output" heading, even when custom scrollbar spacing is applied. Investigate and resolve the Git rebase issues in both the frontend and backend repositories. Merge conflicts should be handled by keeping the upstream implementation of the token tracking feature wherever conflicts occur. Redesign the token usage display to be more subtle and less distracting by moving it into a small footer section at the bottom of the input panel. Add conditional visibility logic for the token counter so that it stays hidden when a new session starts and no tokens have been consumed yet.

| Usage of AI

The AI tools were leveraged as essential partners in this project, primarily for architecture structuring, defining core business logic, and streamlining the development workflow. By automating the translation of schema requirements into actionable code, AI assisted in solving complex technical challenges like topological sorting for foreign key constraints and debugging frontend layout alignment. Furthermore, AI played a key role in iterative refinements, such as implementing token usage tracking and resolving version control conflicts, which significantly accelerated the transition from project planning to a functional implementation.